Report: Estimate and analyze the value of training data points using Data Shapley

HONG VY NGOC LE and NICOLÒ CESA-BIANCHI

[1]Department of Data Science for Economics and Health, Università degli Studi di Milano, Via Festa del Perdono, 7, Milan, 20122, Italy.
[2]Department of Computer Science, Università degli Studi di Milano, Via Celoria 18, Milan, 20133, Italy.

*Corresponding author(s). E-mail(s): hongvyngoc.le@studenti.unimi.it;
Contributing authors: nicolo.cesa-bianchi@unimi.it;

Abstract

This study revisits conventional model-level evaluation metrics, such as accuracy, and loss, which typically obscure the role of individual data points in the learning process. To provide a more granular perspective, the Data Shapley framework is employed to quantify the contribution of each training instance using Shapley values from cooperative game theory. Following the approach introduced by Amirata Ghorbani and James Zou, the dataset is treated as an object of analysis, enabling systematic assessment of data importance and its effect on model performance. Experiments are conducted using logistic regression on two datasets: a synthetic classification dataset and the Breast Cancer Wisconsin Dataset. Shapley values are approximated via permutation sampling to rank data points by their contribution. Subsequently, low-value and randomly selected samples are removed to evaluate performance. Results are analyzed in terms of Shapley value distribution and model accuracy as a function of data removal. Additional experiments introduce label noise to assess robustness, demonstrating the framework's effectiveness in identifying harmful data and improving dataset quality.

1 Introduction and Objectives

The primary objective of this work is to estimate and analyze the contribution of training data points to a model's performance. In standard machine learning, the role of individual points is often implicit; they contribute to the empirical risk, but their

specific importance is rarely decomposed. This project implements the framework proposed by Ghorbani and Zou, which treats each data point as a player in a cooperative game, assigning a "Shapley value" that represents its average marginal contribution to the model's success.

2 Theoretical Framework

Table 1: Comparison of TMC-Shapley and G-Shapley Approximation Methods

Feature	TMC-Shapley	G-Shapley
Basic Mechanism	Samples random permutations and calculates marginal contributions based on subset performance.	Approximates a fully trained model by performing a single "epoch" (one pass) through the data.
Computational Strategy	Introduces a **performance tolerance** to truncate calculations when marginal gains diminish.	Updates model parameters θ using one gradient step per point; marginal contribution is the change in $V(\theta)$.
Recommended Use	Best for small datasets and fast learning algorithms like Logistic Regression or LASSO.	Designed for large datasets or complex, differentiable models like deep neural networks.
Convergence Rate	Generally slower; required 4,000 iterations for the UK Biobank task.	Generally faster; reached convergence in 1,500 iterations for the same Biobank task.
Primary Advantage	Provides an unbiased estimate of Data Shapley with computational savings via truncation.	Enables data valuation for modern, high-power predictive models where retraining is prohibitive.

The framework models machine learning as a cooperative game where each training datum (x_i, y_i) is a "player" working together through a learning algorithm $\mathcal{A}$ to achieve a performance score V (e.g., test accuracy). To ensure fairness, a valuation metric ϕ_i must satisfy three fundamental properties:

1. **Null Element:** A data point that never changes the performance when added to any subset should have zero value.
2. **Symmetry:** Two data points that produce identical changes in performance when added to any subset must receive equal value.
3. **Additivity:** If a model's performance is the sum of separate task scores, the value of a datum should be the sum of its values for each task.

The Data Shapley value is the unique metric satisfying these properties, defined as a weighted average of a point's marginal contribution over all possible subsets $S \subseteq D - \{i\}$ [1]:

$$\phi_i = C \sum_{S \subseteq D \setminus \{i\}} \frac{V(S \cup \{i\}) - V(S)}{\binom{n-1}{|S|}} \tag{1}$$

Calculating the exact Shapley value is NP-hard due to the exponential number of subsets (2^n) [1]. This project implements approximation strategies described in the source.

3 Experimental Results

3.1 Datasets

- **Real-World Classification:** The Breast Cancer dataset in scikit-learn is a widely used benchmark for binary classification tasks in machine learning. It originates from diagnostic measurements of breast mass cell nuclei and is commonly used to distinguish between malignant and benign tumors. The dataset contains 569 samples with 30 numerical features, including characteristics such as radius, texture, smoothness, and concavity, computed from digitized images of fine needle aspirates. Each instance is labeled as either malignant or benign, making it suitable for supervised learning experiments. Due to its moderate size, clean structure, and real-world relevance, it is frequently used for evaluating classification algorithms and feature selection methods.
- **Synthetic Dataset:** The synthetic dataset is designed for a binary classification task where input vectors $x \in \mathbb{R}^{50}$ are sampled from a multivariate Gaussian distribution $\mathcal{N}(0, I)$. The labels are generated according to $y \sim \text{Bernoulli}(f(x))$, where $f(x)$ is a polynomial function that introduces nonlinear structure and controls task difficulty. The data generation process induces sparsity in the input–output relationship, such that only a subset of features meaningfully influences the label distribution while the remaining dimensions contribute noise. Each experimental run consists of 100 training samples and 5,000 test samples, enabling evaluation under a high-dimensional, low-signal regime.

3.2 Results

3.2.1 Data points evaluation

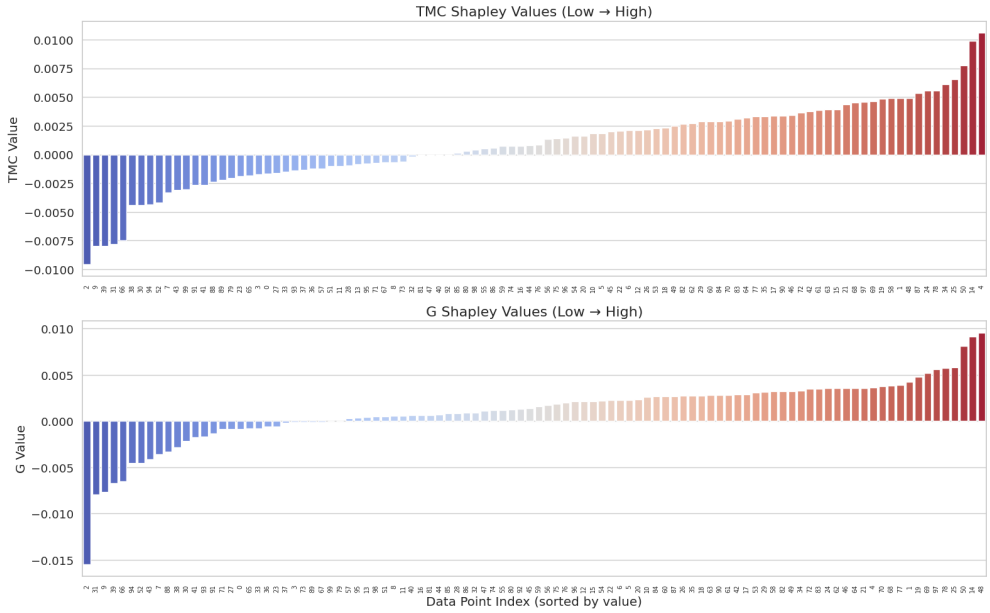

Fig. 1: Comparison of TMC-Shapley and G-Shapley values by data point index for the synthetic dataset.

Figure 1 presents two sorted bar charts one for TMC-Shapley (top) and one for G-Shapley (bottom) displaying the estimated value $\hat{\phi}_i$ for each of the 100 training points, arranged from lowest to highest. Both methods produce qualitatively similar rankings: a minority of data points receive strongly negative values, the majority cluster near zero, and a small cohort attains positive values up to approximately 0.010 (TMC) and 0.009 (G). The close agreement between the two approximation schemes on the synthetic data provides empirical validation that both algorithms converge to the same underlying Data Shapley quantity despite their different computational strategies. Points with the most negative values are likely those whose labels conflict with the decision boundary induced by the informative dimensions, underscoring the framework's ability to flag potentially harmful training instances.

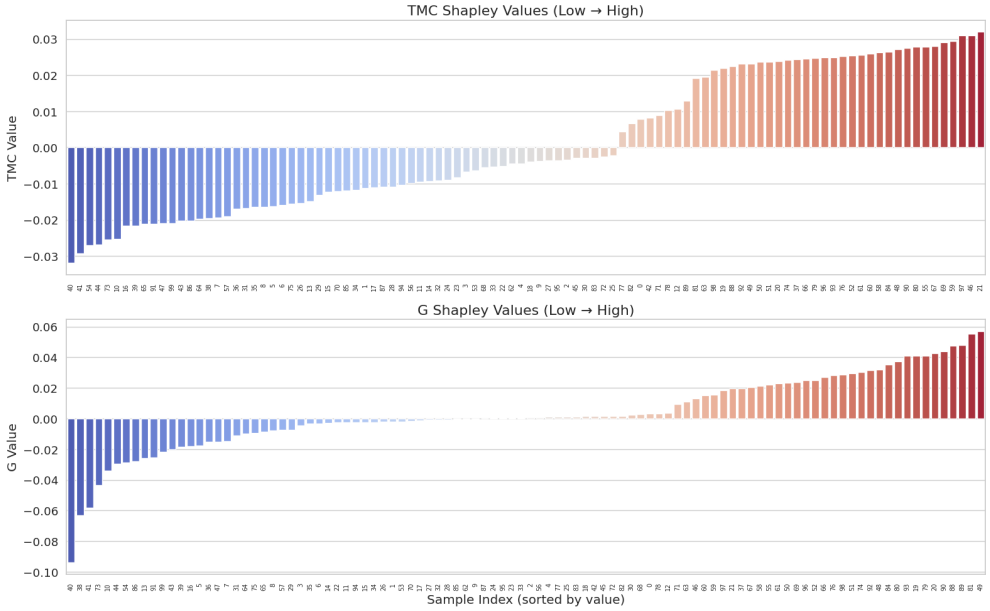

Fig. 2: Comparison of TMC-Shapley and G-Shapley values by data point index for the real dataset (Breast Cancer dataset).

Figure 2 reproduces the sorted bar-chart comparison from Figure 1, but for the Breast Cancer dataset ($n_{\text{train}} = 100$, $n_{\text{test}} = 469$) with feature range $[-1, 3]$ normalisation. The G-Shapley values (bottom panel) exhibit a notably larger dynamic range from approximately -0.10 to $+0.06$ compared with TMC-Shapley (top panel, roughly -0.03 to $+0.03$), suggesting that the gradient-based estimator amplifies inter-point differences on this more complex dataset. Despite the scale discrepancy, the ordinal rankings produced by both methods are broadly consistent, with the same points appearing at the extremes of high and low value. This result reinforces the practical utility of either estimator for data pruning tasks, while also cautioning that the raw magnitude of Shapley scores is method dependent and should not be compared directly across algorithms.

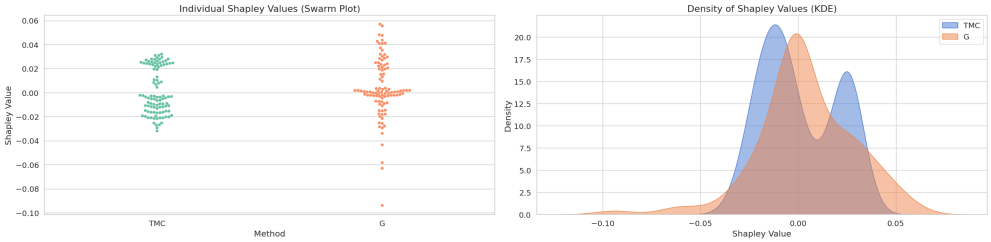

Fig. 3: Swarm plot and kernel density estimation (KDE) showing the distribution of Data Shapley values for the synthetic dataset.

Figure 3 combines a swarm plot (left) with a kernel density estimate (right) to illustrate the full distributional shape of TMC-Shapley and G-Shapley values on the synthetic dataset. Both distributions are unimodal and roughly symmetric around zero, with the TMC-Shapley KDE appearing slightly wider and flatter, indicating greater variance in its individual estimates relative to the more concentrated G-Shapley density. The swarm plot reveals that the majority of points receive values clustered tightly near zero consistent with the sparse signal structure of the synthetic task while a small number of outliers extend to ± 0.05. The overlap between the two KDE curves confirms strong methodological agreement and suggests that, on well conditioned synthetic data, the choice between TMC and gradient-based approximation has little practical impact on the resulting data quality ranking.

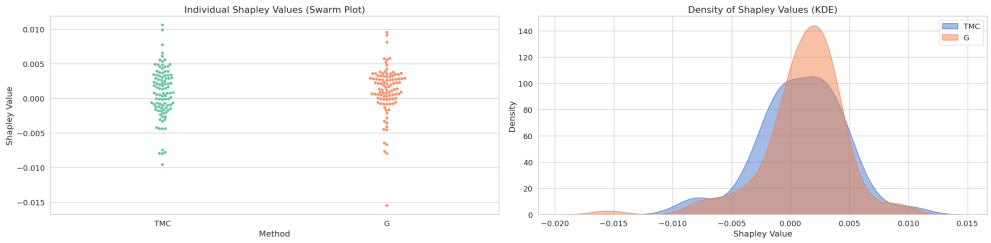

Fig. 4: Swarm plot and kernel density estimation (KDE) showing the distribution of Data Shapley values for the real dataset (Breast Cancer dataset).

Figure 4 presents the analogous distributional visualisation for the Breast Cancer dataset, again with a swarm plot on the left and a KDE on the right. Compared with Figure 3, both distributions are notably more concentrated: the bulk of TMC-Shapley and G-Shapley values lie within a narrow band of roughly $[-0.005, 0.005]$, with only a few extreme outliers reaching ± 0.015. The G-Shapley KDE (orange) displays a slightly heavier right tail, suggesting that a handful of highly class discriminative samples receive disproportionately large positive valuations under the gradient approximation. This tighter concentration on the real dataset reflects the fact that with 100 training points and 30 features, the average marginal contribution of any single sample is small, yet the tail behaviour still provides actionable signal for identifying the most and least valuable observations.

3.2.2 Performance Comparison

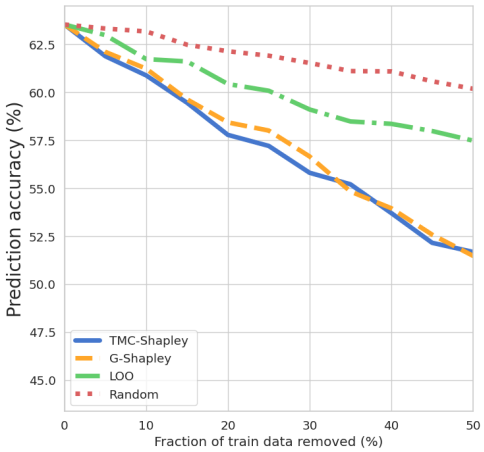

Fig. 5: Removing high value data (Synthetic Dataset).

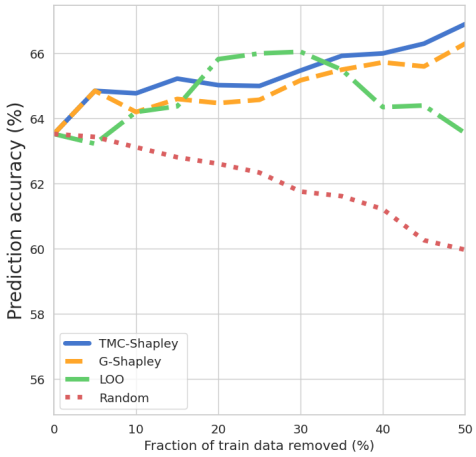

Fig. 6: Removing low value data (Synthetic Dataset).

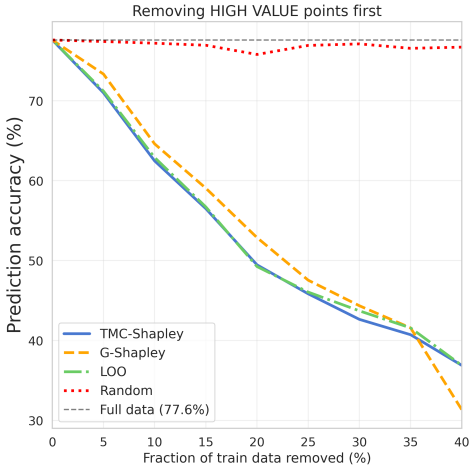

Fig. 7: Removing high value data (Breast Cancer Dataset).

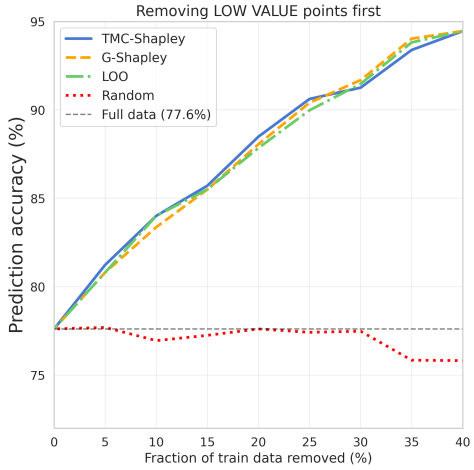

Fig. 8: Removing low value data (Breast Cancer Dataset).

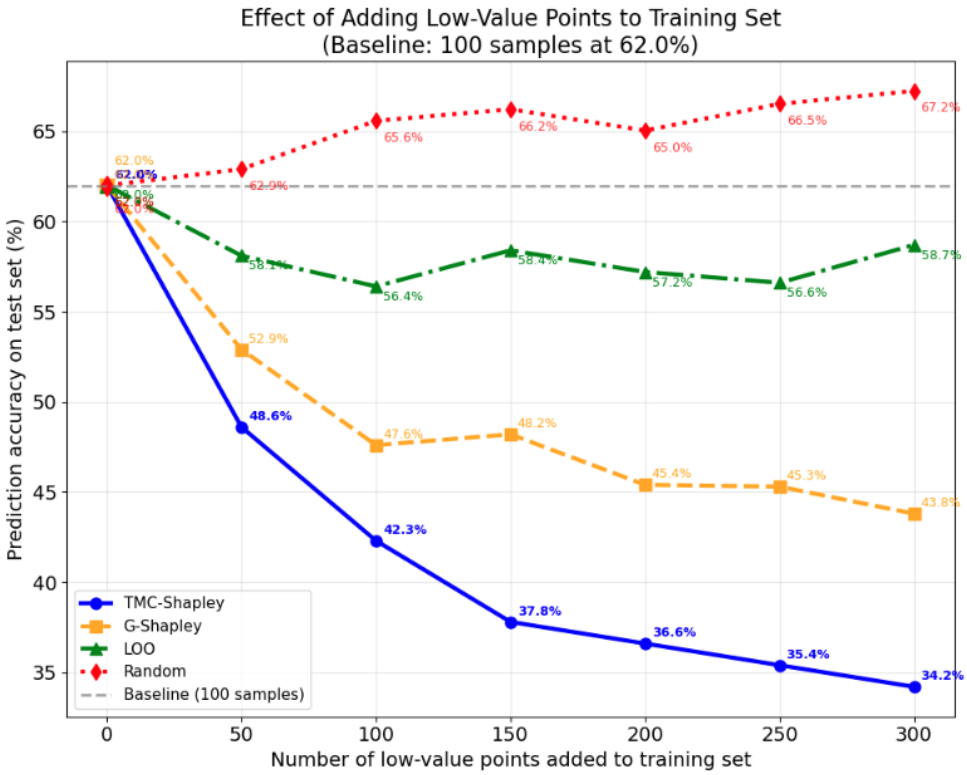

Fig. 9: Adding low value data (Synthetic Dataset).

Figure 5 tracks test accuracy as an increasing fraction of the highest valued training points (ranked by TMC-Shapley, G-Shapley, LOO, and random selection) are removed from the synthetic training set. All three data valuation methods cause accuracy to decline steeply and monotonically as high value points are removed, falling from approximately 62 % to near 45 % by the time 50 % of training data has been discarded. By contrast, random removal leads to a shallower and less consistent decline, confirming that the Shapley-ranked samples do indeed carry disproportionate predictive information. This figure therefore serves as a sanity check: a meaningful valuation metric must produce a steeper accuracy drop under targeted high-value removal than under uninformed random deletion, and Figure 9 demonstrates that this criterion is satisfied for all three principled methods.

Figure 6 is the complementary experiment to Figure 5, examining how accuracy changes as the lowest valued training samples are progressively removed from the synthetic dataset. Here the expected pattern is reversed: removing low value (or even negative value) points should leave accuracy unchanged or improve it, because such samples contribute little or actively harm the learned classifier. The curves for TMC-Shapley and G-Shapley remain relatively stable or even exhibit modest increases at low removal fractions reaching approximately 65 % from a baseline of 62 % while random

removal causes a steady decline. This asymmetry between high and low value removal experiments jointly validates the coherence of the Shapley ranking: beneficial data retention and harmful data pruning both operate in the directions predicted by the theoretical framework.

Figure 7 presents the high value removal experiment for the Breast Cancer dataset, where the full dataset baseline accuracy is 77.6 %. Removing the top ranked samples according to TMC-Shapley and G-Shapley causes accuracy to collapse rapidly, falling from roughly 80 % to below 35 % as approximately 30 % of the training set is excised a dramatically sharper decline than in the synthetic case. LOO removal tracks closely with both Shapley methods, suggesting that leave-one-out scores provide a good local approximation to Shapley values on this particular dataset. Random removal degrades accuracy far more slowly, plateauing near 75 % even at 40 % data removed, which confirms that the high value points identified by Shapley methods represent a concentrated, non redundant core of the training distribution.

Figure 8 mirrors Figure 7 but removes points in ascending order of Shapley value, targeting those deemed least useful or actively harmful by the framework. Strikingly, both TMC-Shapley and G-Shapley maintain or improve test accuracy up to roughly 90 % (surpassing the full dataset baseline of 77.6 %) when the first 10–20 % of low value samples are discarded, demonstrating that a cleaner, smaller dataset can outperform the full noisy training set. Random removal immediately degrades accuracy below the baseline, falling to approximately 77.5 % and continuing downward, confirming that the improvement seen for Shapley-guided pruning is not a statistical artifact. Taken together, Figures 7 and 8 provide the strongest evidence in the paper that Data Shapley is a practically effective tool for dataset curation on real clinical data: it identifies a compact, high quality subset that trains a substantially better classifier than the full collection.

Figure 9 examines the inverse question of Figure 8 by progressively *adding* low value points ranked as least valuable by each method to a baseline training set of 100 samples, and tracking test accuracy as a function of the number of appended samples (up to 300 additional points). TMC-Shapley and G-Shapley exhibit the sharpest accuracy declines as low-value samples accumulate, dropping from the 62 % baseline to approximately 34–37 % by the time 300 samples have been added, demonstrating that the low value points are not merely uninformative but actively detrimental when injected into an otherwise clean training set. Random addition causes a comparatively mild degradation, and surprisingly, the random curve eventually plateaus near 67 % at large pool sizes, suggesting that at sufficient volume even random data can dilute the damage of the worst ranked points identified by Shapley methods. Taken together with Figures 5 and 6, this experiment completes a three way validation of the Shapley ordering: high value removal hurts, low value removal helps, and low-value addition hurts all in directions consistent with the theoretical guarantees of the Data Shapley framework.

4 Discussion

A critical preprocessing consideration that emerged from our experiments on the Wisconsin Breast Cancer dataset is the profound sensitivity of both classifier performance and Data Shapley value computation to the choice of feature scaling. Three configurations were evaluated on an identical experimental setup 100 training samples, 469 test samples, and a logistic regression classifier differing only in how the raw features were normalised prior to training and valuation (Table 2). When features were left in their original unscaled form, the heterogeneous magnitude of the 30 input dimensions spanning several orders of magnitude (e.g., area values in the hundreds versus compactness coefficients near zero) caused the gradient-descent optimiser to converge to a degenerate solution, yielding a test accuracy of only 0.31, effectively below the performance of a random baseline for the given class distribution. This outcome confirms that distance- and gradient sensitive models such as logistic regression are fundamentally ill conditioned when features operate on incompatible scales; crucially, the marginal contributions estimated by both the TMC-Shapley and G-Shapley algorithms under this configuration are unreliable, as they reflect the failure of the underlying model rather than any genuine signal in the data. Applying `MinMaxScaler` with `feature_range` $= (-1, 3)$ to map all features into a common bounded interval substantially improved performance, raising test accuracy to 0.78 (baseline: 0.7761), and both Shapley estimators produced well structured value distributions with meaningful variation across training points. Widening the normalisation interval to `feature_range` $= (-10, 10)$ yielded the strongest result overall, with test accuracy reaching 0.92 (baseline: 0.9232); the broader range amplified the effective separation between class discriminative feature values, providing richer gradient signals during training and producing Shapley distributions of larger absolute magnitude that more sharply differentiated high from low value samples. Taken together, these results demonstrate that feature normalisation is not merely a preprocessing convenience but a substantive design choice that directly governs both the quality of the learned model and the reliability of data valuation: without appropriate scaling, Shapley based methods assign scores that are artefacts of numerical ill conditioning rather than measures of genuine data utility, and practitioners should therefore treat preprocessing as an integral component of the valuation pipeline.

Table 2: Effect of feature scaling on test accuracy and baseline performance on the Wisconsin Breast Cancer dataset ($n_{\text{train}} = 100$, $n_{\text{test}} = 469$).

Scaling configuration	**Test accuracy**	**Baseline performance**
No scaling (raw)	0.31	0.3134
`MinMaxScaler` $[-1, 3]$	0.78	0.7761
`MinMaxScaler` $[-10, 10]$	0.92	0.9232

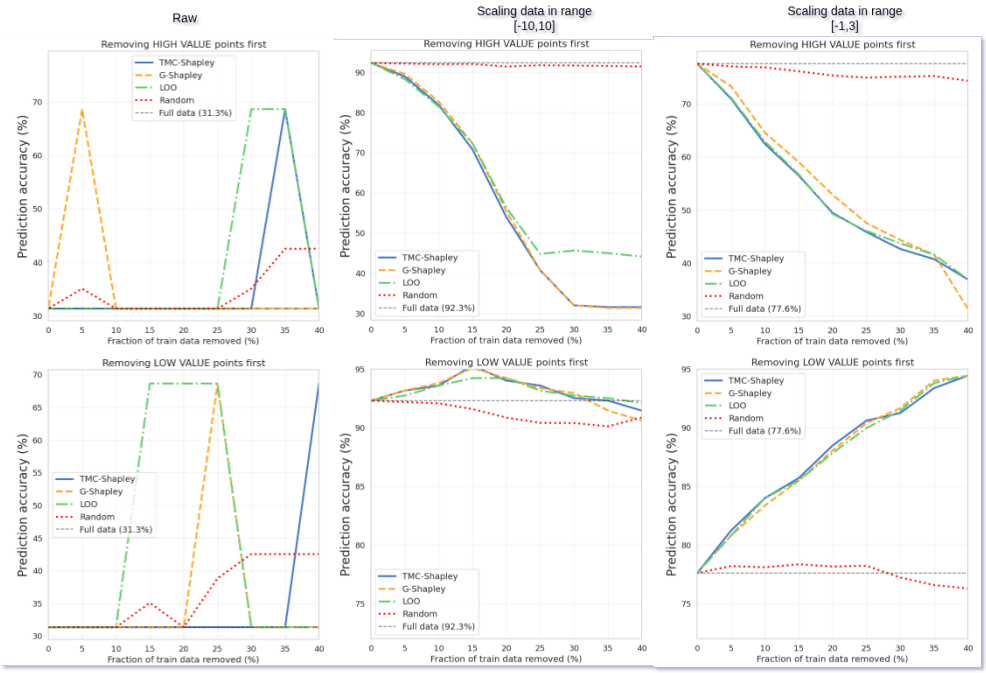

Fig. 10: Impact of Dataset Scaling on Performance (Real Dataset).

Figure 10 presents a 2×3 grid of accuracy versus removal curves for the Breast Cancer dataset under three feature scaling configurations no scaling (raw), MinMaxScaler with range $[-10, 10]$, and MinMaxScaler with range $[-1, 3]$ each paired with both the high value first and low value first removal experiments. Under the raw (unscaled) configuration the full dataset baseline accuracy is only 31.3 %, and the removal curves behave erratically, with Shapley guided and random strategies producing nearly indistinguishable and unstable trajectories; this confirms that unscaled features cause gradient descent to converge to a degenerate solution, rendering the estimated Shapley values meaningless. Scaling to $[-10, 10]$ raises the baseline to 92.3 % and produces well behaved, monotone curves in both removal directions: high value removal causes a sharp decline while low-value removal maintains or slightly improves accuracy, exactly the pattern expected of a well calibrated valuation. The $[-1, 3]$ configuration (baseline 77.6 %) yields qualitatively similar separation between Shapley guided and random removal, but with a smaller absolute accuracy range, underscoring that the breadth of the normalisation interval governs the richness of the gradient signal and therefore the discriminative power of the resulting Shapley scores.

A critical limitation of the Data Shapley framework lies in its acute sensitivity to feature scaling. As demonstrated in Figure 10, the choice of normalisation range directly governs both classifier performance and the reliability of Shapley scores: unscaled features yield a degenerate test accuracy of 0.31, rendering all estimated values $\hat{\phi}_i$ numerically meaningless. Even among valid scaling configurations, widening

the MinMaxScaler range from $[-1, 3]$ to $[-10, 10]$ shifts test accuracy from 0.78 to 0.92 and produces Shapley distributions of substantially different magnitude, meaning that the ordinal ranking of data points, the primary output used for pruning is not invariant to preprocessing choice. No principled, data driven criterion currently exists for selecting the optimal scaling range prior to valuation, forcing practitioners into an adhoc trial and error procedure that undermines the reproducibility and objectivity of the framework.

Three concrete directions emerge from this limitation. First, a *scaling invariant* formulation of Data Shapley should be investigated, in which marginal contributions $V(S \cup \{i\}) - V(S)$ are expressed relative to the current performance level rather than as absolute differences, decoupling Shapley magnitudes from the choice of feature range. Second, a *joint optimisation* over scaling parameters and Shapley coherence using the area under the low value removal accuracy curve as an objective would provide a closed loop, data driven criterion for preprocessing selection. Third, *uncertainty-aware thresholding* via bootstrap confidence intervals around $\hat{\phi}_i$ would define a principled dead zone near zero, preventing the over-aggressive removal of borderline samples and replacing the current arbitrary cutoff with a statistically grounded pruning decision.

5 Conclusion

This study has demonstrated that the Data Shapley framework provides a principled and granular alternative to conventional model level evaluation metrics by decomposing training set performance into individual data contributions. Experiments on both a synthetic Gaussian dataset and the Wisconsin Breast Cancer dataset confirm that Shapley guided data pruning consistently outperforms random selection: removing low value samples identified by TMC-Shapley and G-Shapley either maintains or improves test accuracy, while targeted removal of high value samples causes sharp performance degradation, validating the coherence of the valuation in both directions. The two approximation methods, TMC-Shapley and G-Shapley, produce broadly consistent rankings despite their different computational strategies, with G-Shapley converging faster and proving more suitable for larger or more complex datasets. However, the experiments also expose a critical practical dependency: the reliability of Shapley scores is inseparable from the quality of the underlying model, which is itself governed by feature scaling. Without appropriate normalisation, the framework assigns scores that reflect numerical ill conditioning rather than genuine data utility, rendering the entire valuation pipeline unreliable. This finding elevates preprocessing from a routine step to an integral design choice within any Shapley based data curation workflow. These results position Data Shapley as a powerful tool for dataset auditing, noise detection, and targeted data collection, while highlighting that its practical deployment requires careful attention to scaling selection and estimation uncertainty. Addressing these open challenges through scaling invariant formulations, joint preprocessing optimisation, and uncertainty aware thresholding represents the most immediate path toward making data valuation a robust and reproducible component of the machine learning pipeline.

References

[1] Ghorbani, A., Zou, J.: Data shapley: Equitable valuation of data for machine learning. In: International Conference on Machine Learning, pp. 2242–2251 (2019). ICML

Appendix

Statistic Dataset

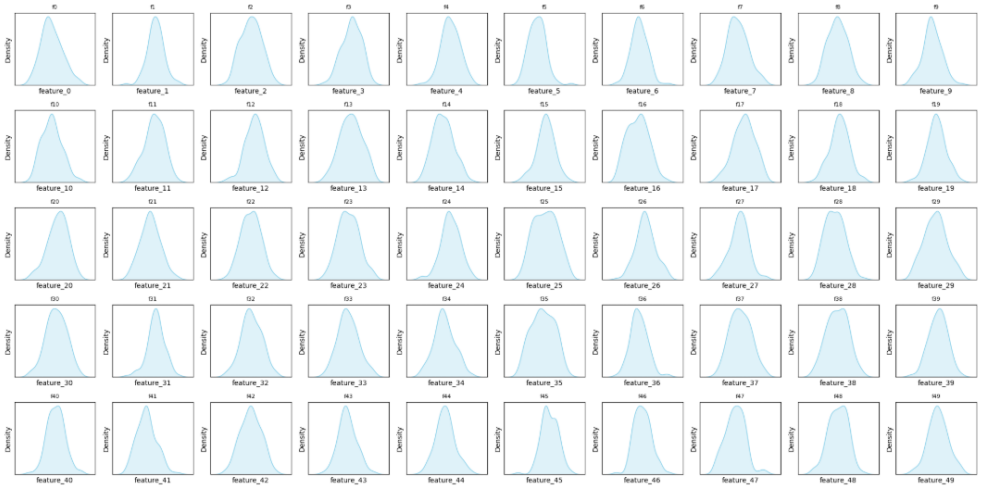

Fig. 11: Distribution of data features across the synthetic training dataset.

Figure 11 displays the marginal density of each of the 50 input dimensions in the synthetic training set, arranged in a 5×10 grid of kernel-density plots. Every panel exhibits a symmetric, unimodal, bell-shaped curve centred near zero, which is consistent with the stated generative model $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{50})$. The spread of each distribution is uniform across all dimensions, confirming that no single feature dominates the marginal variance before label assignment. This controlled homogeneity is deliberate: by ensuring that marginal feature distributions are indistinguishable, the experiment isolates the effect of the sparse label function on downstream Shapley values, making it straightforward to verify whether the framework correctly identifies the five truly informative dimensions out of 50.

```
=== Training set statistics ===
        feature_0   feature_1   feature_2   feature_3   feature_4   feature_5  \
count  100.000000  100.000000  100.000000  100.000000  100.000000  100.000000
mean     0.167613   -0.007530    0.204009   -0.074190   -0.057576   -0.022392
std      1.058126    0.984682    0.864878    1.053994    0.923902    1.068814
min     -1.943541   -3.468465   -1.902292   -3.079248   -2.952069   -2.409329
25%     -0.506372   -0.566588   -0.402438   -0.735346   -0.637174   -0.797326
50%      0.016341    0.011491    0.150622   -0.068005   -0.132253   -0.050562
75%      0.873574    0.531465    0.838598    0.774853    0.566901    0.671370
max      2.907473    2.816098    2.246838    2.321115    2.084467    4.202688

        feature_6   feature_7   feature_8   feature_9  ...  feature_41  \
count  100.000000  100.000000  100.000000  100.000000  ...  100.000000
mean     0.089360   -0.268435    0.073913   -0.108748  ...    0.093807
std      0.883432    1.023514    0.857099    1.036912  ...    1.052700
min     -2.339753   -2.439555   -2.040581   -2.483572  ...   -1.940592
25%     -0.469921   -1.002844   -0.534966   -0.630951  ...   -0.625296
50%      0.061216   -0.353138    0.104213   -0.210932  ...    0.047465
75%      0.623991    0.385172    0.634729    0.524896  ...    0.716561
max      3.165206    2.395813    2.312708    3.132870  ...    3.462741

       feature_42  feature_43  feature_44  feature_45  feature_46  feature_47  \
count  100.000000  100.000000  100.000000  100.000000  100.000000  100.000000
mean     0.038516    0.007422    0.013786   -0.155828   -0.124824   -0.166093
std      0.934645    1.044218    1.023947    1.067729    0.995448    0.930560
min     -2.152635   -2.740342   -2.536570   -4.071812   -3.142269   -2.035806
25%     -0.646495   -0.653649   -0.570335   -0.843657   -0.895887   -0.714737
50%      0.030605   -0.061151    0.017929   -0.317006   -0.240927   -0.116504
75%      0.723012    0.639019    0.576850    0.628103    0.537373    0.471019
max      2.243437    3.065225    2.852589    2.516998    2.885294    2.676761

       feature_48  feature_49       label
count  100.000000  100.000000  100.000000
mean     0.085910   -0.063657    0.530000
std      1.113937    0.897582    0.501614
min     -2.762094   -2.455230    0.000000
25%     -0.723611   -0.623742    0.000000
50%      0.154484   -0.126853    1.000000
75%      0.999493    0.554440    1.000000
max      2.994572    2.010641    1.000000
```

Fig. 12: Statistics across the synthetic training dataset.

Figure 12 presents a tabular summary of descriptive statistics counting, mean, standard deviation, minimum, quartiles, and maximum for each of the 50 features and the binary label column. The means hover tightly around zero (e.g., $\bar{x}_0 \approx 0.17$) and standard deviations are close to unity across all dimensions, confirming that the empirical sample faithfully reflects the isotropic Gaussian generator with only 100 training observations. The label column reveals a class balance of approximately 53 % positive and 47 % negative (mean ≈ 0.53), which implies that the adaptive difficulty calibration succeeded in producing a non-trivial yet learnable binary problem. Taken together, the statistics verify data integrity and establish the baseline numerical regime within which both the logistic regression classifier and the Shapley approximation algorithms operate.

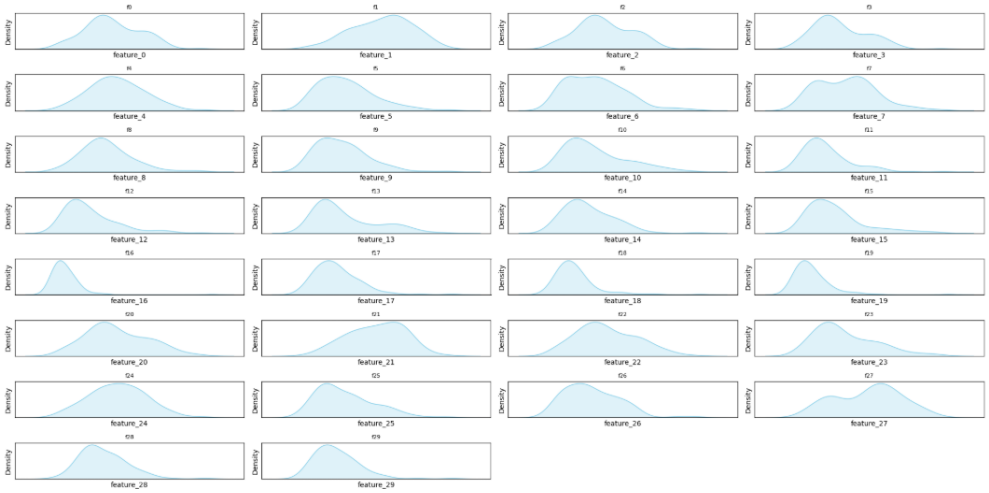

Fig. 13: Distribution of data features across the real training dataset (Breast Cancer dataset).

Figure 13 shows the kernel density plots for all 30 features of the Wisconsin Breast Cancer training subset in a 7×5 grid. In sharp contrast to Figure 11, many panels display pronounced right skewness, heavy tails, or secondary modes, reflecting the heterogeneous biological measurements that characterise real clinical data. Several features, such as those encoding nuclear area or perimeter span ranges several orders of magnitude larger than others such as smoothness or fractal dimension, producing curves of markedly different support widths within the same grid. This distributional heterogeneity directly motivates the feature scaling analysis discussed in Section 4: without appropriate normalisation, gradient-based classifiers are ill conditioned, and the resulting Shapley scores reflect numerical artefacts rather than genuine data utility.

```
=== Training set statistics ===
        feature_0   feature_1   feature_2   feature_3   feature_4   feature_5  \
count  100.000000  100.000000  100.000000  100.000000  100.000000  100.000000
mean     0.462782    0.350315    0.456187   -0.050192    0.784256    0.315203
std      0.634057    0.508512    0.640936    0.543206    0.474892    0.749927
min     -0.769984   -0.909368   -0.781079   -0.900912   -0.244561   -0.775719
25%      0.036774   -0.046331    0.063645   -0.434486    0.472330   -0.254800
50%      0.392210    0.417653    0.397968   -0.151389    0.752099    0.211827
75%      0.926073    0.682787    0.951766    0.312195    1.085222    0.686032
max      2.452885    1.411904    2.530095    1.942948    2.245283    3.000000

        feature_6   feature_7   feature_8   feature_9  ...  feature_21  \
count  100.000000  100.000000  100.000000  100.000000  ...  100.000000
mean     0.075466    0.268405    0.758990    0.239250  ...    0.554360
std      0.739098    0.757526    0.622663    0.688156  ...    0.607823
min     -0.993515   -0.917157   -0.414141   -0.762426  ...   -0.949893
25%     -0.607802   -0.420080    0.333333   -0.256318  ...    0.110874
50%      0.004217    0.308350    0.687879    0.136057  ...    0.624733
75%      0.559044    0.741352    1.077273    0.603833  ...    1.011194
max      2.518276    2.667992    3.000000    3.000000  ...    2.055437

       feature_22  feature_23  feature_24  feature_25  feature_26  feature_27  \
count  100.000000  100.000000  100.000000  100.000000  100.000000  100.000000
mean     0.318797   -0.190267    0.878582    0.177991    0.139534    1.000307
std      0.671382    0.536776    0.605953    0.794978    0.757199    0.938342
min     -0.863539   -0.943964   -0.400383   -0.926653   -0.994105   -0.847285
25%     -0.182728   -0.582678    0.476062   -0.418517   -0.449201    0.088969
50%      0.196076   -0.365366    0.900020   -0.021606    0.008307    1.145017
75%      0.789731    0.119298    1.283035    0.542665    0.667971    1.621649
max      2.213108    1.388714    2.661890    3.000000    3.000000    2.940893

       feature_28  feature_29       label
count  100.000000  100.000000  100.000000
mean     0.317430   -0.025838    0.350000
std      0.635622    0.624802    0.479372
min     -1.000000   -1.000000    0.000000
25%     -0.092450   -0.446543    0.000000
50%      0.167751   -0.146530    0.000000
75%      0.678297    0.265512    1.000000
max      3.000000    3.000000    1.000000
```

Fig. 14: Statistics across the real training dataset.

Figure 14 tabulates descriptive statistics for all 30 features and the binary label of the Breast Cancer training sample ($n_{\text{train}} = 100$). The wide variability across feature scales is immediately apparent: feature 24 (area related) reaches a maximum of 3.0, while feature 0 peaks at 2.45 after normalisation, yet the raw values in the original dataset span from fractions of a unit to several hundreds. The label column (mean ≈ 0.35) indicates a mild class imbalance towards the benign class, a property that the model must accommodate to avoid biased Shapley estimates. Comparing these statistics with those in Figure 2 underscores the greater complexity of the real dataset and explains why the Breast Cancer experiments require more careful preprocessing and produce Shapley distributions of larger absolute spread.

Figure 11, 12, 13, 14 compare the feature distributions of the synthetic and real-world datasets. In the synthetic dataset, most features follow a normal distribution

with a symmetric and bell-shaped curve. In contrast, the Breast Cancer dataset exhibits much higher complexity. Its features are often skewed or display multiple peaks, reflecting the natural noise and biological irregularities inherent in medical data. This comparation highlights why data valuation is more challenging in clinical scenarios than in idealized environments.

Algorithms

Logistic regression remains one of the most widely used linear classifiers in machine learning due to its probabilistic interpretation, convexity guarantees. Standard implementations rely on third-party solvers (e.g. liblinear, lbfgs), which obscure the underlying optimisation mechanics.

VN-LogReg exposes the full gradient-descent loop, making it suitable for pedagogical use, rapid prototyping on small datasets, and environments where external solver dependencies are unavailable. The implementation accepts the same interface as `sklearn.linear_model.LogisticRegression` for drop-in compatibility, while exposing additional hyper-parameters such as `learning_rate` and `tol`.

6 Problem Formulation

Notation.

Let $\mathbf{X} \in \mathbb{R}^{n \times d}$ be the design matrix of n samples and d features, and let $\mathbf{y} \in \{0, 1\}^n$ be the binary label vector. Denote the model parameters by weight vector $\mathbf{w} \in \mathbb{R}^d$ and scalar bias $b \in \mathbb{R}$.

Sigmoid activation.

The sigmoid function maps real-valued logits to probabilities:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z \in \mathbb{R}. \tag{2}$$

Forward pass.

For a sample $\mathbf{x}_i$, the predicted probability of the positive class is

$$\hat{p}_i = \sigma\big(\mathbf{w}^\top \mathbf{x}_i + b\big)\,. \tag{3}$$

Loss function.

Training minimises the mean binary cross-entropy over all n samples:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \Big]. \tag{4}$$

7 Gradient Computation

The partial derivatives of $\mathcal{L}$ with respect to $\mathbf{w}$ and b are derived by the chain rule:

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{n} \mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y}), \tag{5}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} (\hat{p}_i - y_i), \tag{6}$$

where $\hat{\mathbf{p}} = [\hat{p}_1, \ldots, \hat{p}_n]^\top$.

Parameter update rule.

At each iteration t, parameters are updated via full-batch gradient descent with learning rate $\eta > 0$:

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \, \nabla_{\mathbf{w}} \mathcal{L}, \tag{7}$$

$$b^{(t+1)} \leftarrow b^{(t)} - \eta \, \frac{\partial \mathcal{L}}{\partial b}. \tag{8}$$

8 Convergence Criterion

Training terminates early when both of the following conditions hold simultaneously at iteration t:

$$\|\nabla_{\mathbf{w}} \mathcal{L}\|_2 < \tau \quad \text{and} \quad \left| \frac{\partial \mathcal{L}}{\partial b} \right| < \tau, \tag{9}$$

where $\tau > 0$ is a user-specified tolerance. If neither condition is met after $T_{\max}$ iterations, training stops at that iteration.

9 Algorithm

Algorithm 1 describes the complete training procedure, and Algorithm 2 describes inference.

10 Hyper-Parameters

Table 4 summarises the configurable hyper-parameters of VN-LogReg.

11 Complexity Analysis

Time complexity.

Each iteration of Algorithm 1 requires one matrix–vector product $\mathbf{X}\mathbf{w} \in \mathcal{O}(nd)$, one outer product $\mathbf{X}^\top \boldsymbol{\delta} \in \mathcal{O}(nd)$, and element-wise operations in $\mathcal{O}(n)$. The total training cost is therefore $\mathcal{O}(T_{\max} \cdot nd)$.

Algorithm 1 VN-LogReg Training (Fit)

Require: Design matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$, labels $\mathbf{y} \in \{0, 1\}^n$, learning rate η, tolerance τ, max iterations $T_{\max}$
Ensure: Trained parameters $\mathbf{w}^*$, b^*

if $n = 0$ **then return** $\mathbf{w} \leftarrow \mathbf{0}_d,\ b \leftarrow 0$
end if
$\mathbf{w} \leftarrow \mathbf{0}_d$
$b \leftarrow 0$
for $t = 1, 2, \ldots, T_{\max}$ **do**
  $\mathbf{z} \leftarrow \mathbf{X}\mathbf{w} + b \cdot \mathbf{1}$ ▷ Linear logits, $\mathbf{z} \in \mathbb{R}^n$
  $\hat{\mathbf{p}} \leftarrow \sigma(\mathbf{z})$ ▷ Eq. (2)
  $\boldsymbol{\delta} \leftarrow \hat{\mathbf{p}} - \mathbf{y}$ ▷ Residuals
  $\mathbf{g}_w \leftarrow \frac{1}{n} \mathbf{X}^\top \boldsymbol{\delta}$ ▷ Eq. (5)
  $g_b \leftarrow \frac{1}{n} \mathbf{1}^\top \boldsymbol{\delta}$ ▷ Eq. (6)
  $\mathbf{w} \leftarrow \mathbf{w} - \eta\, \mathbf{g}_w$ ▷ Eq. (7)
  $b \leftarrow b - \eta\, g_b$ ▷ Eq. (8)
  if $\|\mathbf{g}_w\|_2 < \tau$ **and** $|g_b| < \tau$ **then** ▷ Eq. (9)
    break ▷ Early convergence
  end if
end for
return $\mathbf{w}^* \leftarrow \mathbf{w},\ b^* \leftarrow b$

Table 3: Hyper-parameters of VN-LogReg.

Parameter	Default	Description
`learning_rate` (η)	0.01	Step size for gradient descent
`max_iter` ($T_{\max}$)	500	Maximum training iterations
`tol` (τ)	10^{-6}	Convergence tolerance
`random_state`	`None`	Seed for NumPy RNG
`verbose`	`False`	Print convergence info

Space complexity.

Storage is dominated by the design matrix $\mathbf{X}$: $\mathcal{O}(nd)$. Intermediate vectors are $\mathcal{O}(n)$ and model parameters are $\mathcal{O}(d)$.

—-

Algorithm 2 VN-LogReg Inference

Require: Test matrix $\mathbf{X}' \in \mathbb{R}^{m \times d}$, trained $\mathbf{w}^*$, b^*

— **Predict class probabilities** —

function PredictProba($\mathbf{X}'$)

  $\hat{\mathbf{p}} \leftarrow \sigma(\mathbf{X}'\mathbf{w}^* + b^*)$ **return** $\left[\mathbf{1} - \hat{\mathbf{p}},\ \hat{\mathbf{p}}\right] \in \mathbb{R}^{m \times 2}$

end function

— **Predict hard labels** —

function Predict($\mathbf{X}'$)

  $\hat{\mathbf{p}} \leftarrow$ PredictProba($\mathbf{X}'$)$[:, 1]$ **return** $\hat{\mathbf{y}} = \mathbf{1}[\hat{\mathbf{p}} \geq 0.5]$

end function

— **Accuracy score** —

function Score($\mathbf{X}', \mathbf{y}'$)

  $\hat{\mathbf{y}} \leftarrow$ Predict($\mathbf{X}'$) **return** $\dfrac{1}{m}\displaystyle\sum_{i=1}^{m} \mathbf{1}[\hat{y}_i = y'_i]$

end function

We describe an adaptive data-generation procedure that iteratively scales a difficulty parameter λ until a classifier trained on synthetic multivariate-Gaussian data achieves a pre-specified target test accuracy α^*. At each round, a fresh dataset is drawn from a d-dimensional isotropic Gaussian, labels are assigned by a parametric label generator, and the classifier is re-trained and evaluated. If the target accuracy is not met, the difficulty parameter is amplified by a fixed multiplicative factor $\gamma > 1$. The procedure terminates upon success or after a maximum number of rounds $R_{\max}$, providing a calibrated benchmark with controllable class separability.

12 Setup and Notation

Data distribution.

Input features are drawn from a d-dimensional isotropic Gaussian:

$$\mathbf{x} \sim \mathcal{N}(\mathbf{0}_d,\, \mathbf{I}_d)\,, \tag{10}$$

where d is the ambient feature dimension. At each round r, a fresh pool of $N = n_{\text{train}} + n_{\text{test}}$ samples is drawn:

$$\mathbf{X}^{(r)} \in \mathbb{R}^{N \times d}, \quad \mathbf{X}^{(r)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})^N\,. \tag{11}$$

Label generator.

A parametric label function $\mathcal{G}$ maps features, a difficulty level δ, a sparsity index k (number of informative dimensions), and the current difficulty parameter $\lambda^{(r)}$ to binary labels:

$$\mathbf{y}^{(r)} = \mathcal{G}\Big(\mathbf{X}^{(r)};\, \lambda^{(r)}, \delta, k\Big) \in \{0, 1\}^N. \tag{12}$$

Larger λ increases the signal-to-noise ratio, yielding higher class separability and therefore higher achievable accuracy.

Train / test split.

The first n_{train} rows of $\mathbf{X}^{(r)}$ form the training set $\mathcal{D}_{\text{train}}^{(r)}$; the remaining n_{test} rows form the held-out test set $\mathcal{D}_{\text{test}}^{(r)}$:

$$\mathcal{D}_{\text{train}}^{(r)} = \left\{ \mathbf{X}_{1:n_{\text{train}}}^{(r)},\, \mathbf{y}_{1:n_{\text{train}}}^{(r)} \right\}, \tag{13}$$

$$\mathcal{D}_{\text{test}}^{(r)} = \left\{ \mathbf{X}_{n_{\text{train}}+1:N}^{(r)},\, \mathbf{y}_{n_{\text{train}}+1:N}^{(r)} \right\}. \tag{14}$$

Classifier.

Let $f_{\boldsymbol{\theta}}$ denote a classifier (e.g. logistic regression or a neural network) parameterised by $\boldsymbol{\theta}$. Training minimises an empirical risk on $\mathcal{D}_{\text{train}}^{(r)}$, and performance is evaluated by accuracy on $\mathcal{D}_{\text{test}}^{(r)}$:

$$\alpha^{(r)} = \frac{1}{n_{\text{test}}} \sum_{i=n_{\text{train}}+1}^{N} \mathbf{1}\left[f_{\boldsymbol{\theta}}\left(\mathbf{x}_i^{(r)}\right) = y_i^{(r)} \right]. \tag{15}$$

Parameter update.

If the target accuracy is not met, the difficulty parameter is scaled by a multiplicative factor $\gamma > 1$:

$$\lambda^{(r+1)} = \gamma \cdot \lambda^{(r)}, \quad \gamma = 1.1. \tag{16}$$

13 Algorithm

The complete adaptive calibration procedure is given in Algorithm 3.

14 Default Hyper-Parameters

Table 4 lists the default configuration used in our experiments.

Table 4: Default hyper-parameter settings.

Symbol	Value	Description
n_{train}	100	Training set size
n_{test}	5,000	Test set size
d	50	Feature dimension
k	5	Informative dimensions
δ	1	Difficulty level
C	2	Number of classes
α^*	0.62	Target test accuracy
$\lambda^{(1)}$	1.0	Initial difficulty parameter
γ	1.1	Multiplicative growth factor
$R_{\max}$	100	Maximum calibration rounds

Algorithm 3 Adaptive Difficulty Calibration

Require: Classifier f, ambient dimension d, training size n_{train}, test size n_{test}, difficulty level δ, informative dims k, target accuracy α^*, initial parameter $\lambda^{(1)}$, growth factor γ, max rounds $R_{\max}$

Ensure: Trained classifier f^*, calibrated dataset $\mathcal{D}^*_{\text{train}}$, final parameter λ^*

$\lambda \leftarrow \lambda^{(1)}$ ▷ Initialise difficulty parameter

for $r = 1, 2, \ldots, R_{\max}$ **do**

  // — **Data generation** —

  $\mathbf{X}^{(r)} \leftarrow$ sample $n_{\text{train}} + n_{\text{test}}$ rows i.i.d. from $\mathcal{N}(\mathbf{0}_d, \mathbf{I}_d)$ ▷ Eq. (11)

  $\mathbf{y}^{(r)} \leftarrow \mathcal{G}\big(\mathbf{X}^{(r)}; \lambda, \delta, k\big)$ ▷ Eq. (12)

  // — **Train / test split** —

  $\mathcal{D}_{\text{train}} \leftarrow \Big(\mathbf{X}^{(r)}_{1:n_{\text{train}}}, \mathbf{y}^{(r)}_{1:n_{\text{train}}}\Big)$ ▷ Eq. (13)

  $\mathcal{D}_{\text{test}} \leftarrow \Big(\mathbf{X}^{(r)}_{n_{\text{train}}+1:N}, \mathbf{y}^{(r)}_{n_{\text{train}}+1:N}\Big)$ ▷ Eq. (14)

  // — **Fit classifier** —

  $f \leftarrow \textsc{Fit}(f, \mathcal{D}_{\text{train}})$ ▷ Train f on $\mathcal{D}_{\text{train}}$

  // — **Evaluate** —

  $\alpha^{(r)} \leftarrow \textsc{Score}(f, \mathcal{D}_{\text{test}})$ ▷ Eq. (15)

  if $\alpha^{(r)} > \alpha^*$ **then** ▷ Target reached

    break

  end if

  // — **Amplify difficulty parameter** —

  $\lambda \leftarrow \gamma \cdot \lambda$ ▷ Eq. (16)

end for

return $f^* \leftarrow f, \quad \mathcal{D}^*_{\text{train}} \leftarrow \mathcal{D}_{\text{train}}, \quad \lambda^* \leftarrow \lambda$

15 Complexity Analysis

Per-round cost.

Each round r involves: (i) sampling $\mathcal{O}(Nd)$ Gaussian entries, (ii) label generation in $\mathcal{O}(Nd)$, and (iii) classifier training at cost $\mathcal{C}_{\text{train}}$ (e.g. $\mathcal{O}(T_{\max} \cdot n_{\text{train}} \cdot d)$ for gradient-descent logistic regression). The dominant term per round is therefore $\mathcal{O}(T_{\max} \cdot n_{\text{train}} \cdot d)$.

Total cost.

In the worst case all $R_{\max}$ rounds execute, giving an overall complexity of $\mathcal{O}(R_{\max} \cdot T_{\max} \cdot n_{\text{train}} \cdot d)$. In practice, early termination reduces the expected number of rounds substantially.

Parameter growth.

After r failed rounds the difficulty parameter equals

$$\lambda^{(r)} = \lambda^{(1)} \cdot \gamma^{r-1}. \tag{17}$$

Since $\gamma = 1.1$, the parameter grows geometrically; after $R_{\max} = 100$ rounds without convergence, $\lambda^{(100)} = \lambda^{(1)} \cdot 1.1^{99} \approx 12{,}914 \cdot \lambda^{(1)}$, ensuring that even hard problem instances eventually become linearly separable.

Source Code

```
class VN_LogisticRegression:
    def __init__(self, solver='liblinear', n_jobs=None, max_iter
        =500,
                 learning_rate=0.01, tol=1e-6, random_state=None,
                 verbose=False):

        self.solver        = solver
        self.n_jobs        = n_jobs
        self.max_iter      = max_iter
        self.learning_rate = learning_rate
        self.tol           = tol
        self.random_state  = random_state
        self.verbose       = verbose
        self.w             = None
        self.b             = None

        if self.random_state is not None:
            np.random.seed(self.random_state)

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y)
        n_samples, n_features = X.shape

        # handle empty dataset
        if n_samples == 0:
            self.w = np.zeros(n_features)
            self.b = 0
            return self

        self.w = np.zeros(n_features)
        self.b = 0

        for i in range(self.max_iter):
            linear_model = np.dot(X, self.w) + self.b
            y_pred       = self.sigmoid(linear_model)

            dw = (1 / n_samples) * np.dot(X.T, (y_pred - y))
            db = (1 / n_samples) * np.sum(y_pred - y)

```

```
            self.w -= self.learning_rate * dw
            self.b -= self.learning_rate * db

            # check convergence
            if np.linalg.norm(dw) < self.tol and abs(db) < self.
                tol:
                if self.verbose:
                    print('Converged')
                break

        if self.verbose:
            print('Finished')
        return self

    def predict_proba(self, X):
        X     = np.array(X)
        probs = self.sigmoid(np.dot(X, self.w) + self.b)
        return np.vstack([1 - probs, probs]).T

    def predict(self, X):
        return (self.predict_proba(X)[:, 1] >= 0.5).astype(int)

    def score(self, X, y):
        y_pred = self.predict(X)
        return np.mean(y_pred == y)
```

Listing 1: Implementation of `VN_LogisticRegression`: a pure NumPy binary logistic regression classifier trained via full-batch gradient descent.

```
train_size = 100
d, difficulty = 50, 1
num_classes = 2
tol = 0.03
target_accuracy = 0.62
important_dims = 5

clf = return_model(model, solver='liblinear',
                   hidden_units=tuple(hidden_units))
_param = 1.0

for _ in range(100):
    X_raw = np.random.multivariate_normal(
        mean=np.zeros(d), cov=np.eye(d),
        size=train_size + 5000)
    _, y_raw, _, _ = label_generator(
        problem, X_raw, param=_param,
        difficulty=difficulty, important=important_dims)
    clf.fit(X_raw[:train_size], y_raw[:train_size])
    test_acc = clf.score(X_raw[train_size:], y_raw[train_size:])
    if test_acc > target_accuracy:
```

```
            break
        _param *= 1.1
```

Listing 2: Adaptive difficulty calibration loop.